

Overview

🎯 In this course, you will learn about Squish extensions and how they are used to recognize Qt Quick Control and custom components.

[Customizing QtQuick support in Squish](#)
[Qt Convenience API | Squish 9.2](#)

What will you learn

1. How to use a Squish extension to recognize components from Qt Quick Control
2. How to create and install a Squish extension.
3. How to use a Squish extension to recognize custom components.

Need of Squish Extension

A Squish Extension acts as a translator between Squish and complex custom UI elements inside your application under test (AUT).

There are two completely different ways to make user interfaces in Qt:

1. The OLD way = Qt Widgets - No extensions needed

In traditional Qt C++ development, a button is a native, hard-coded object in the system called a `QPushButton`. When Squish looks at a traditional app, the application explicitly broadcasts, "I am a Button object!" Squish understands this natively. No extensions are required.

2. The NEW way = Qt Quick / QML

Modern Qt uses a language called **QML** to draw fluid, mobile-style interfaces. This is what your tutorial is using.

In QML, a "Button" is not a hard-coded system object. Instead, QML builds everything out of basic, primitive shapes. Think of them like **Lego blocks**. A QML button is actually just three primitive blocks stacked on top of each other:

1. A `Rectangle` (to draw the colored background).
2. A `Text` element (to write the label, like "Submit").
3. A `MouseArea` (an invisible box placed on top that listens for clicks).

⚠ Squish extension is required for QT Quick /QML based applications

Installing a Squish Extension

A squish extension file is a `.qml` file with a SquishHook component as its top-level component. When starting a test, Squish will automatically check the folder `SQUISHDIR/lib/extensions/qt/qtquick` for Qt Quick extension files.

 SQUISHDIR stands for the directory where the Squish package is installed.

So, an extension can be installed by simply placing an extension file in that folder. Additional directories can also be checked by setting the environment variable `SQUISH_QTQUICK_EXTENSION_PATH`. Squish environment variable can be modified by going to the “Test Suite Settings” and navigating the “AUT” tab.

An example of an extension file already present in every Squish installation is

`ControlsExt.qml`.

When installing Squish for Qt, an extension for Qt Quick Controls is automatically installed to the directory at `SQUISHDIR/lib/extensions/qt/qtquick`. Here a file called `ControlsExt.qml` can be found. This file allows Squish to recognize all the components which are part of Qt Quick Controls.

If we remove this file then any application that is built using QML / Qt Quick , it will not be able to properly record each element and thus fail during running.

Creating a Squish Extension

Custom Squish extensions can be created and installed so that Squish will recognize custom components. A Squish extension in QML starts with import statements for Qt Quick and `com.froglogic.squish.squish.qtquick`. This second import statement will import the module containing the `SquishHook` component which is the root component used in a Squish extension.

None

```
import QtQuick 2.0
import com.froglogic.squish.qtquick 0.1
```

SquishHook provides properties and functions which define how Squish should identify a component.

SquishHook Properties

1. `blacklistedTypes`
Specifies a list of strings, which are QML types to be ignored by Squish.
2. `containerTypes`
Specifies a list of strings, which are type names of classes which can be containers for all their children.
3. `priority`
Holds a priority value for the QML extension. Squish access QML extensions in priority order, from highest to lowest. Its default value is zero.

None

```
priority: 110
```

4. `objectNameProperties`
A dictionary of key-value pairs. The key is the QML type name, and the value are the properties Squish will use in the real name.

None

```
readonly property var objectNameProperties: {
    "Button": [ "text" ]
}
```

5. `whitelistedTypes`

Specifies a list of strings, which are QML types to be noticed by Squish.

None

```
readonly property vat whitelistedTypes: [ "Button" ]
```

SquishHook Overridable Functions

1. `containerFor(item)`
returns the container the given item is in or null if the item is not in a container.
2. `extraPropertiesMor(item)`
returns a list of the property names that should be taken into account when generating a Squish object name for the item. This function is functionally the same as using `objectNameProperties` or modifying the Qt wrapper descriptor file but allows for finer control.
3. `isItemReady(item)`
returns true if the given item is visible, enabled, and not being or about to be animated.
4. `propertyFor(item, property)`
returns the property value for the given item's property (specified as a string).
5. `isIgnored(item)`
returns false if the given item should be noticed by Squish.

None

```
function isIgnored(item) {  
    if ( item.parent && qmlType(item.parent) = "Button" ) {  
        return true  
    }  
    return unhandled  
}
```

SquishHook Helper Functions

1. `isCustomQMLType(item)`
returns whether the given item is a custom type.
2. `isQMLType(item, className)`
returns whether the given item inherits the class specified by `className`.
3. `itemHasContents(item)`

returns whether the given item has the `QQuickItem::ItemHasContents` flag set.

4. `qmlId(item)`
returns the id for the given item or an empty string if item has no id set.
5. `qmlType(item)`
returns the type of item if it is a declarative type or an empty string.
6. `source(item)`
returns the given item of type `Item`'s URL if the item was loaded from a local file or remotely; otherwise, it returns the item's containing component's URL.

Requirements for Creating an Extension for a Custom Component

An extension for a custom component should at least define the `whitelistedTypes` property to ensure the custom component is noticed by Squish. The `priority` property must also be appropriately set to ensure that the correct extension is used in case several extensions are installed. Then, additional properties and overridable functions should be defined based on the functionality of the custom component. For example, if the AUT contains a custom button which is made up of a `Text` component and a `Rectangle` component, the `isIgnored()` function should be used to ignore the `Text` and `Rectangle`, and the name of the custom button should be added to the `whitelistedTypes` property. An example of an extension for this situation is shown in the code snippet below.

None

```
SquishHook {
    priority: 110;
    readonly property var whitelistedTypes: [ "CustomButton" ];
    readonly property var objectNameProperties: {
        "CustomButton": [ "text" ],
    }

    function isIgnored(item) {
        if ( item.parent && qmlType( item.parent ) ==
"CustomButton" ) {
            return true;
        }
        else if ( item.parent.parent && qmlType( item.parent.parent
) == "CustomButton" ) {
```

```
        return true;
    }
    return unhandled;
}
}
```

With the extension written and installed, Squish will recognize the component using the name defined in the `whitelistedTypes` property and use it during the generation of the object map.

Steps

Summary

Squish's recognition of QML components

Squish recognizes all components which subclass `QObject` in C++. Therefore all Qt Widgets components are recognized by Squish by default. Squish's recognition of QML components functions slightly differently. When testing a Qt Quick application, Squish will, by default, recognize components that are part of the Qt Quick module. Squish will also recognize components which are part of Qt Quick Controls thanks to a pre-installed extension. However, Squish will not recognize components that are part of any other module, or any custom components. For this reason, if no extensions are used, Squish will not be able to generate an object map based on custom components, or modules other than Qt Quick or Qt Quick Controls.

Location of the Qt Quick extension files

A Squish extension file is simply a .qml file which has a `SquishHook` component as its top-level component. When starting a test, Squish will automatically check the folder `SQUISHDIR/lib/extensions/qt/qtquick` for Qt Quick extension files. So, an extension can be installed by simply placing an extension file in that folder.

When installing Squish for Qt, an extension for Qt Quick Controls is automatically installed to the directory at `SQUISHDIR/lib/extensions/qt/qtquick`. Here a file called `ControlsExt.qml` can be found. This file allows Squish to recognize all the components which are part of Qt Quick Controls.

With custom extensions Squish will recognize custom components

Creating a custom Squish extension starts by creating a new QML file. A Squish extension in QML starts with import statements for **Qt Quick** and **com.froglogic.squish.qtquick**. This second import statement will import the module containing the `SquishHook` component which is the root component used in a Squish extension. `SquishHook` provides properties and functions which define how Squish should identify a component.

An extension for a custom component should at least define the

whitelistedTypes property to ensure the custom component is noticed by Squish. The **priority** property must also be appropriately set to ensure that the correct extension is used in case several extensions are installed. Then, additional properties and overridable functions should be defined based on the functionality of the custom component.

With the extension written and installed, Squish will recognize the component using the name defined in the **whitelistedTypes** property and use it during the generation of the object map.